

Chapter 8

Virtual memory

Every process believes it owns a huge, private address space starting at 0. In reality its pages are scattered over physical memory, many of them are not there at all, and some are shared with other processes. The MMU translates every single memory access, and the kernel keeps the translation tables. This chapter covers the translation (x86-64's four-level page tables, the TLB), the page fault as the kernel's hook into it (demand paging, copy-on-write), and what happens when physical memory runs out (page replacement).

8.1 Why virtual memory

- **Isolation:** a process can only reach the frames its page tables map; the kernel's pages are mapped without the user bit.
- **Relocation:** every program can be linked for the same addresses.
- **Overcommit:** memory is promised (`mmap` of 1 GiB) and delivered page by page on first touch; unused parts cost nothing.
- **Sharing:** the same physical frame can appear in several address spaces (shared libraries, copy-on-write after `fork`).

8.2 Paging and the x86-64 page tables

Memory is divided into 4 KiB *pages* (virtual) and *frames* (physical). A page table maps page numbers to frame numbers. A flat table for a 48-bit address space would have 2^{36} entries of 8 bytes — 512 GiB per process. Instead, x86-64 uses a tree of four levels; each table is one 4 KiB frame with 512 entries, and only the parts of the tree that map something exist.

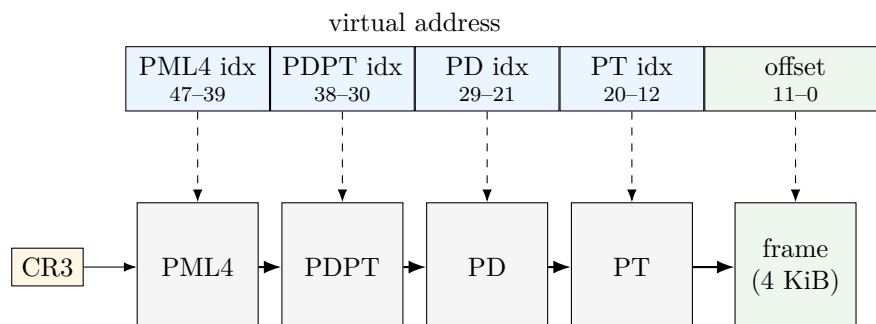


Figure 8.1: A page walk: each 9-bit index selects one of 512 entries; the entry holds the frame number of the next table, and finally of the page.

A page-table entry (64 bits) holds the frame number (bits 12–51) and flags, among them:

Bit	Name	Meaning
0	P (present)	the entry is valid; otherwise any access faults
1	W (writable)	stores allowed
2	U (user)	accessible from ring 3
5	A (accessed)	set by the MMU on every access
6	D (dirty)	set by the MMU on every write
9–11	available	ignored by the MMU, free for the OS (e.g. a COW mark)
63	NX	no instruction fetch

The effective rights are the AND over all four levels: the upper levels are usually created permissive (P, W, U) and the leaf entry decides. Mapping one page into an empty address space costs three new tables (PDPT, PD, PT); unmapping should free tables that became empty.

8.3 The TLB

A page walk costs four extra memory accesses. The *translation lookaside buffer* caches recent translations (page number → frame, rights) and makes almost all accesses hit. The TLB is not coherent with the page tables: when the kernel changes or removes a present entry, it must invalidate the cached copy (`invlpg addr`, or reload `cr3` to drop everything). Forgetting it leaves the old translation usable — after an unmap, a program keeps reading a frame that may already belong to someone else. Entries that were not present are never cached, so *adding* a mapping needs no invalidation. On multi-core machines other CPUs may cache the translation too: a *TLB shutdown* sends them an interrupt to invalidate — expensive, and batched by real kernels.

8.4 Page faults

When the MMU cannot translate (not present) or the rights do not allow the access (write to read-only, user access to kernel page), it raises a page fault: the CPU stores the faulting address (x86: in `cr2`) and an error code (present?, write?, user?) and enters the kernel’s handler. The handler decides:

- legal but not loaded yet: allocate/load the page, map it, return — the CPU *re-executes* the faulting instruction;
- copy-on-write page written: copy it, map the copy writable, return;
- anything else: the program made a mistake — signal `SIGSEGV` (Linux) or end the process (SWEB).

Demand paging. Nothing is loaded before it is used. Code and data are read from the executable on the first access; heap and stack pages are allocated on first touch and **zero-filled** — a frame straight from the free pool may contain another process’s data. On Linux even reading an untouched anonymous page only maps a shared zero page.

Copy-on-write. `fork` shares all frames between parent and child and makes every writable page read-only in *both*, with a software “COW” mark. The first write from either side faults; the handler copies the frame and maps the copy writable. If the frame has only one user left (reference count 1), it simply becomes writable again. Note that the parent’s TLB may still hold writable entries for those pages — they must be invalidated, or the parent’s writes land in the shared frame.

8.5 Page replacement

When no frame is free, a page must be evicted (written to swap if dirty) to make room. Which one?

OPT (Belady) evict the page whose next use lies furthest in the future. Optimal, but needs the future; the yardstick for the others.

FIFO evict the page loaded longest ago. Simple, often bad, and it suffers from *Belady's anomaly*: more frames can cause more faults.

LRU evict the page unused for the longest time. Good, but exact LRU needs a timestamp per access — too expensive in hardware.

Clock (second chance) approximates LRU with the accessed bit: a hand sweeps over the frames; a page with $R = 1$ gets its bit cleared and a second chance, the first page with $R = 0$ is evicted.

Worked example, reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1, 3 frames: FIFO 15 faults, LRU 12, OPT 9, Clock 14. Belady's anomaly with FIFO on 1 2 3 4 1 2 5 1 2 3 4 5: 9 faults with 3 frames, 10 with 4. LRU and OPT are *stack algorithms* — the pages kept with k frames are always a subset of those kept with $k + 1$ — and never show the anomaly. (module 08, part C simulates all four with exact rules.)

Working set and thrashing. The *working set* is the set of pages a process used in the recent past. If the working sets of all running processes do not fit into memory, every process keeps faulting out the pages the others need: the system is busy paging and does no useful work — *thrashing*. The remedy is fewer processes in memory (suspend some), not a faster disk.

In SWEB

arch/x86/64/source/ArchMemory.cpp: `mapPage` walks the four levels and allocates missing tables (P, W, U on the upper levels); `unmapPage` frees tables that became empty; frames come from `PageManager::allocPPN`. The kernel reaches any frame through the identity mapping at `IDENT_MAPPING_START` (`getIdentAddressOfPPN`). Page faults go to `common/source/mm/PageFaultHandler.cpp`; a valid user fault calls `Loader::loadPage`, which only knows the ELF segments — a fault elsewhere, e.g. just below the single stack page, ends the process. For P1 this matters twice: every new thread needs its own user stack somewhere in the shared address space, and two threads of one process can now fault at the same time — `mapPage` has no lock of its own.

Key points

- Four levels of 512-entry tables; the leaf decides the rights, the MMU ANDs all levels.
- Changing or removing a present PTE requires a TLB invalidation.
- Page faults are the kernel's hook: demand paging (zero-filled!), copy-on-write, or kill.
- COW: read-only + mark in both address spaces, copy on first write, no copy for the last user.
- $\text{OPT} \leq \text{LRU}$; FIFO shows Belady's anomaly; Clock approximates LRU with the accessed bit.
- Thrashing: working sets do not fit; reduce the load.